Python Programming – Case Studies

10 Programs with Sample Outputs

Note: The sample data, fees, thresholds, and policies below are illustrative.

1. University Scholarship Allocation

```
students = [
    ("Sita", 88, 82, 90, 30000),
    ("Ravi", 76, 75, 88, 25000),
    ("Anu", 85, 78, 86, 35000),
    ("Kiran", 92, 65, 95, 20000)
]

eligible = []
for name, marks, subject, attendance, income in students:
    if marks >= 80 and subject >= 70 and attendance >= 85 and income <= 40000:
        eligible.append((name, marks, subject, income))

eligible.sort(key=lambda s: (-s[1], -s[2], s[3]))

print("SCHOLARSHIP RANKING")
for rank, s in enumerate(eligible, 1):
    print("Rank:", rank, "| Name:", s[0], "| Marks:", s[1])
print("INELIGIBLE STUDENTS")
for s in students:
    if not any(e[0] == s[0] for e in eligible):
        print(s[0], "- Not eligible")
```

Sample Output:

```
SCHOLARSHIP RANKING
Rank: 1 | Name: Sita | Marks: 88
Rank: 2 | Name: Anu | Marks: 85
INELIGIBLE STUDENTS
Ravi - Not eligible
Kiran - Not eligible
```

2. Progressive Electricity Billing

```
def calculate_bill(units):
    if units <= 100:
        amount = units * 2
    elif units <= 200:
        amount = 100 * 2 + (units - 100) * 3
    else:
        amount = 100 * 2 + 100 * 3 + (units - 200) * 4
    return amount + 50

usage = [100, 150, 250]
bills = []
for month, units in enumerate(usage, 1):
    total = calculate_bill(units)
    bills.append(total)
    print("Month", month, "- Units:", units, "- Bill: Rs.", total)

for i in range(1, len(bills)):
    increase = bills[i] - bills[i-1]
    if increase > 200:
        print("Unusual increase in month", i+1, ": Rs.", increase)
```

Sample Output:

```
Month 1 - Units: 100 - Bill: Rs. 250
Month 2 - Units: 150 - Bill: Rs. 400
Month 3 - Units: 250 - Bill: Rs. 800
Unusual increase in month 3 : Rs. 400
```

3. Hackathon Team Formation and Ranking

```
teams = [
    ("Team A", [80, 90]),
    ("Team B", [85, 85]),
    ("Team C", [90, 85]),
    ("Team D", [70])
]

valid = []
for name, scores in teams:
    if 2 <= len(scores) <= 5:
        valid.append((name, sum(scores)))

valid.sort(key=lambda t: (-t[1], t[0]))

print("HACKATHON RANKING")
for rank, (name, score) in enumerate(valid, 1):
    print("Rank:", rank, "|", name, "| Score:", score)
print("Invalid team: Team D (team size below 2)")
```

Sample Output:

```
HACKATHON RANKING
Rank: 1 | Team C | Score: 175
Rank: 2 | Team A | Score: 170
Rank: 3 | Team B | Score: 170
Invalid team: Team D (team size below 2)
```

4. Hospital Billing and Patient Classification

```
patients = [
    ("Ravi", 1500, 2000, 1000, True),
    ("Anu", 1000, 1000, 500, False)
]

for name, consultation, diagnostic, treatment, insured in patients:
    total = consultation + diagnostic + treatment
    discount = total * 0.20 if insured and total >= 3000 else 0
    final_bill = total - discount
    category = "High payment" if final_bill >= 3000 else "Normal payment"
    print(name, "- Total:", int(total), "| Discount:", int(discount),
          "| Final:", int(final_bill), "|", category)
```

Sample Output:

```
Ravi - Total: 4500 | Discount: 900 | Final: 3600 | High payment
Anu - Total: 2500 | Discount: 0 | Final: 2500 | Normal payment
```

5. Multi-Plan Telecommunications Cost Optimization

```
plans = [
    ("Plan A", 199, 100, 28),
    ("Plan B", 299, 200, 28),
    ("Plan C", 399, 300, 56)
]
usage, required_days, budget, extra_rate = 180, 28, 350, 2
eligible = []

for name, price, data, validity in plans:
    extra = max(0, usage - data) * extra_rate
    total = price + extra
    print(name, "- Cost:", total, "| Validity:", validity, "days")
    if validity >= required_days and total <= budget:
        eligible.append((total, name))

if eligible:
    cost, name = min(eligible)
    print("Most economical eligible plan:", name, "| Cost:", cost)
else:
    print("No eligible plan within budget")
```

Sample Output:

```
Plan A - Cost: 359 | Validity: 28 days
Plan B - Cost: 299 | Validity: 28 days
Plan C - Cost: 399 | Validity: 56 days
Most economical eligible plan: Plan B | Cost: 299
```

6. Railway Fare and Reservation Calculator

```
passengers = [
    ("Ravi", 30, 300, 2),
    ("Anu", 10, 300, 2),
    ("Ram", 65, 500, 4)
]
reservation_fee = 40
cancellation_rate = 0.20

for name, age, rate, distance_units in passengers:
    fare = rate * distance_units
    if age < 12:
        fare *= 0.50
    elif age >= 60:
        fare *= 0.70
    total = fare + reservation_fee
    penalty = total * cancellation_rate
    print(name, "- Fare:", int(total), "| Cancellation penalty:", int(penalty))
```

Sample Output:

```
Ravi - Fare: 640 | Cancellation penalty: 128
Anu - Fare: 340 | Cancellation penalty: 68
Ram - Fare: 1440 | Cancellation penalty: 288
```

7. Campus Event Financial Planning

```
venue, equipment, transport = 10000, 5000, 3000
catering, certificate = 200, 50
sponsorship, fee, participants = 10000, 300, 100

fixed_cost = venue + equipment + transport
total_cost = fixed_cost + participants * (catering + certificate)
income = sponsorship + participants * fee
shortfall = max(0, total_cost - income)
surplus_per_registration = fee - catering - certificate

break_even = "Not possible"
if surplus_per_registration > 0:
    break_even = max(0, (fixed_cost - sponsorship + surplus_per_registration - 1)
                     // surplus_per_registration)

print("Total cost:", total_cost)
print("Total income:", income)
print("Sponsorship shortfall:", shortfall)
print("Break-even registrations:", break_even)
```

Sample Output:

```
Total cost: 43000
Total income: 40000
Sponsorship shortfall: 3000
Break-even registrations: 160
```

8. Examination Result and Grace-Mark Validation

```
marks = [38, 28, 65, 70]
pass_mark = 35
grace_limit = 5
maximum_mark = 100

final_marks = []
for mark in marks:
    if mark < pass_mark and pass_mark - mark <= grace_limit:
        mark = min(pass_mark, maximum_mark)
    final_marks.append(mark)

total = sum(final_marks)
percentage = total / len(final_marks)
result = "Pass" if all(m >= pass_mark for m in final_marks) else "Fail"

print("Final marks:", final_marks)
print("Total:", total)
print("Percentage:", percentage, "%")
print("Result:", result)
```

Sample Output:

```
Final marks: [38, 35, 65, 70]
Total: 208
Percentage: 52.0 %
Result: Pass
```

9. Attendance Recovery Computation

```
attended, conducted = 36, 50
required = 75

extra = 0
while attended / conducted * 100 < required:
    attended += 1
    conducted += 1
    extra += 1

print("Current attendance: 36/50 =", round(36/50*100, 2), "%")
print("Extra consecutive classes needed:", extra)
print("Final attendance:", attended, "/", conducted)
print("Final percentage:", round(attended/conducted*100, 2), "%")
```

Sample Output:

```
Current attendance: 36/50 = 72.0 %
Extra consecutive classes needed: 6
Final attendance: 42 / 56
Final percentage: 75.0 %
```

10. ATM Transaction Sequence Simulation

```
balance = 5000
transactions = [1000, -2000, -500, -5000, 1500]
max_withdrawal = 3000
successful = rejected = 0

for amount in transactions:
    if amount >= 0:
        balance += amount
        successful += 1
    else:
        withdrawal = abs(amount)
        if withdrawal <= balance and withdrawal <= max_withdrawal:
            balance -= withdrawal
            successful += 1
        else:
            print("Rejected transaction:", amount)
            rejected += 1

print("Final balance:", balance)
print("Successful transactions:", successful)
print("Rejected transactions:", rejected)
```

Sample Output:

```
Rejected transaction: -5000
Final balance: 5000
Successful transactions: 4
Rejected transactions: 1
```